

LMS AI Application — Architecture, Responsibilities & Coding Plan

10-Day Hackathon Guide for the AI + FastAPI + AWS Workstream

1. What the AI Application Is Actually Solving

Core problem: A normal LMS can tell a student their quiz score, but it does not necessarily identify what they failed to understand or what they should do next.

Recommended product framing: The AI system identifies learning gaps, explains the weak concepts using the teacher's course material, provides targeted practice, and helps route the student to the appropriate intervention — AI assistance, peer support, or teacher support.

Do not pitch the system as guaranteeing that AI will make every student 100%. A stronger and more defensible goal is **continuous mastery**: identify gaps, intervene, reassess, and escalate when AI or peer support is not enough.

2. The Three Core AI Capabilities

Capability	User	Purpose	Priority
AI Quiz Generator	Teacher	Course material → generated questions → teacher edits/approves → publish	P0
Learning Gap Analyzer	Student	Quiz results → incorrect questions → weak areas → priority	P0
AI Learning Assistant	Student	Explain weak areas, use course context, provide examples and targeted practice	P0
Student Matching	Student	Match a struggling student with a peer who is strong in the relevant topic	P1
Teacher Intervention	Student/Teacher	Escalate cases where AI/peer support is insufficient	P1

3. Teacher Flow

Each teacher has a dashboard containing their subjects/courses and class information. When a teacher uploads course material, the AI can use that material to generate assessment questions. The teacher remains the authority: generated questions should be reviewed and edited before publication.

Teacher → Dashboard → Subject/Course → Upload Material → AI Quiz Generator → Generated Questions → Teacher Edits/Approves → Publish Quiz → Students Take Quiz

4. Student Flow

Students see their subjects/courses and the quizzes assigned by their teachers. After a quiz, the backend calculates the score and identifies the questions the student answered incorrectly. The AI Learning Assistant is then given the evidence needed to help with those weak areas.

Student → Dashboard → Course → Quiz → Submit Answers → Score + Incorrect Questions → Learning Gap Analysis → AI Learning Assistant → Targeted Practice → Reassessment

5. What the AI Should Have Access To

For the MVP, keep the AI context focused. It does not need unrestricted access to a student's entire LMS profile.

- **Student identifier** — mainly for correlation/auditing; authorization should be handled before the AI service receives data.
- **Subject/course** — tells the AI the learning context.
- **Quiz score/percentage** — indicates overall performance.
- **Incorrect questions** — the most important evidence of weakness.

- **Student's answers** — helps diagnose what the student misunderstood.
- **Correct answers** — provides the target answer for comparison.
- **Topic associated with each question** — enables topic-level learning-gap detection.
- **Relevant course material** — grounds explanations in what the teacher provided.

6. Why the Student's Wrong Answer Matters

Giving the AI only the question is insufficient for diagnosis. The useful minimum context is: **question + student's answer + correct answer + topic + relevant course material**. This allows the system to distinguish simply getting a question wrong from evidence of a particular misconception.

Example: if the question asks for Newton's Second Law, the student answers $F = m/a$, and the correct answer is $F = ma$, the system has evidence about the student's misunderstanding rather than merely knowing that the answer was wrong.

7. AI Should Not Do Everything

Python/deterministic logic: score calculations, thresholds, identifying incorrect answers, grouping errors by topic, ranking learning gaps, and basic matching logic.

LLM/Bedrock: personalized explanations, examples, conversational teaching, targeted practice generation, and teacher/student summaries.

Key principle: Python diagnoses; the LLM teaches and personalizes.

8. The Mastery / Intervention Loop

The strongest product concept is not an AI chatbot. It is a learning-recovery loop:

ASSESS → DIAGNOSE → INTERVENE → REASSESS → MEASURE → ESCALATE IF NECESSARY

A student might first receive AI assistance. If performance remains weak, the platform can recommend a peer who is strong in the relevant topic. If the student continues to struggle, the platform can flag the need for teacher intervention.

9. Student and Teacher Matching

Matching does not need an LLM initially. It can be a deterministic matching engine based on subject/course, topic mastery, class, availability and willingness to help.

Example: Student A has 42% mastery in Newton's Second Law. Student B has 94% mastery in the same topic. The matching engine can rank Student B highly as a peer-support candidate.

Teacher escalation can similarly be triggered when a student repeatedly performs poorly despite AI and/or peer intervention.

10. FastAPI Service Boundaries

NestJS should own the main LMS/backend responsibilities. FastAPI should own the AI/intelligence responsibilities. The frontend should communicate with the main backend rather than directly calling the AI provider.

Frontend → NestJS → FastAPI AI Service → Bedrock / RAG / Recommendation Logic → FastAPI → NestJS → Frontend

11. Initial FastAPI Endpoints

Endpoint	Purpose
GET /health	Service health check

Endpoint	Purpose
POST /ai/analyze	Analyze quiz performance and identify learning gaps
POST /ai/assistant	Provide grounded learning assistance for identified weak areas
POST /ai/quiz/generate	Generate draft questions from teacher course material
POST /ai/practice	Generate targeted practice for a weak topic
POST /ai/match/peer	Find suitable peer support
POST /ai/match/teacher	Recommend/escalate to teacher support

12. Build Order — Do Not Start With RAG

- Step 1 — Define the data contract:** agree with the NestJS team on exactly what FastAPI receives and returns.
- Step 2 — Build /health:** establish the FastAPI service and basic deployment-friendly structure.
- Step 3 — Build /ai/analyze:** use ordinary Python to identify incorrect questions, group by topic, calculate weakness and return learning gaps.
- Step 4 — Connect Bedrock:** use the diagnosed weak area as context for personalized explanations.
- Step 5 — Build targeted practice:** generate questions specifically for the diagnosed weakness.
- Step 6 — Add the AI Assistant:** allow the student to ask follow-up questions about the weak area.
- Step 7 — Add course-material grounding/RAG:** chunk, embed and retrieve relevant material when direct context is no longer sufficient.
- Step 8 — Add peer/teacher matching:** use deterministic matching and escalation rules.
- Step 9 — Docker + AWS:** containerize and deploy to ECR/ECS, add IAM, Secrets Manager and CloudWatch.
- Step 10 — Integration/hardening:** test NestJS ↔ FastAPI ↔ Bedrock and rehearse the learning-improvement demo.

13. Suggested FastAPI Project Structure

```
ai-service/
├── app/
│   ├── main.py
│   └── api/routes/
│       ├── analyze.py
│       ├── assistant.py
│       ├── quiz.py
│       └── schemas/
│           ├── analyze.py
│           ├── assistant.py
│           ├── quiz.py
│           └── services/
│               ├── analyzer.py
│               ├── assistant.py
│               ├── quiz_generator.py
│               └── ai/
│                   ├── bedrock.py
│                   ├── prompts.py
│                   └── core/
│                       ├── config.py
│                       ├── tests/
│                       ├── Dockerfile
│                       ├── requirements.txt
│                       ├── .env.example
│                       └── README.md
```

14. AI + AWS Tooling

Tool/Service	Role
Python	AI service and deterministic learning logic
FastAPI	AI microservice/API
Pydantic	Request/response validation
Boto3	Python access to AWS services, including Bedrock
Amazon Bedrock	Foundation models for generative AI
PostgreSQL + pgvector	Course/AI data and vector search when RAG is added
S3	Teacher course-material storage
Docker	Package FastAPI

Tool/Service	Role
Amazon ECR	Container image registry
Amazon ECS Fargate	Run the AI service
IAM	Least-privilege AWS permissions
Secrets Manager	Application/database secrets
CloudWatch	Logs and monitoring
pytest	Testing
Swagger/FastAPI /docs	API testing/documentation
Git/GitHub	Version control and collaboration

15. What Not to Build First

Do not begin with Kubernetes, fine-tuning, multi-agent systems, complex LangChain/LlamaIndex abstractions, or a large RAG pipeline. These add complexity before the core learning loop is working.

First prove: quiz result → incorrect answer → learning gap → intervention. Everything else should strengthen that loop.

16. The Hackathon Demo Story

The strongest demonstration is a before/after learning story rather than a generic chatbot demonstration.

Student scores 40% → system identifies the weak topic → AI explains the concept using course context → targeted practice is generated → student retakes a short assessment → performance improves → if the student still struggles, the system recommends peer or teacher intervention.

Recommended pitch: “Most LMS platforms tell students what they scored. Our platform tells them what they don’t understand, gives them the next best intervention, and measures whether that intervention worked.”

17. Your Immediate Coding Starting Point

Start with the **Learning Gap Analyzer**. Build it with dummy data first. Do not connect Bedrock or RAG yet.

The first working pipeline should be:

Quiz Result → Incorrect Questions → Group by Topic → Calculate Weakness → Return Learning Gaps + Recommended Intervention

Once this works, Bedrock can be added exactly where generative AI provides value: explaining the identified weakness, adapting the explanation to the learner, and generating targeted practice.